

DATA PIPELINE DESIGN

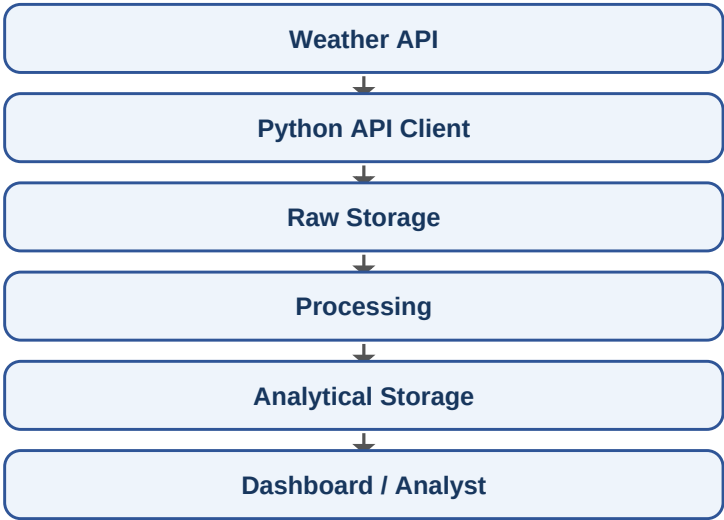
Weather API — A Simple End-to-End Data Pipeline

Selected Data Source	Weather API
Pipeline Type	Batch / Periodic Data Collection
Primary Use Case	Historical weather analysis and dashboard reporting

Objective

Design a simple data pipeline that explains where weather data comes from, how it enters the pipeline, how it is processed, where it is stored, and who uses it.

Pipeline at a Glance



1. Selected Source

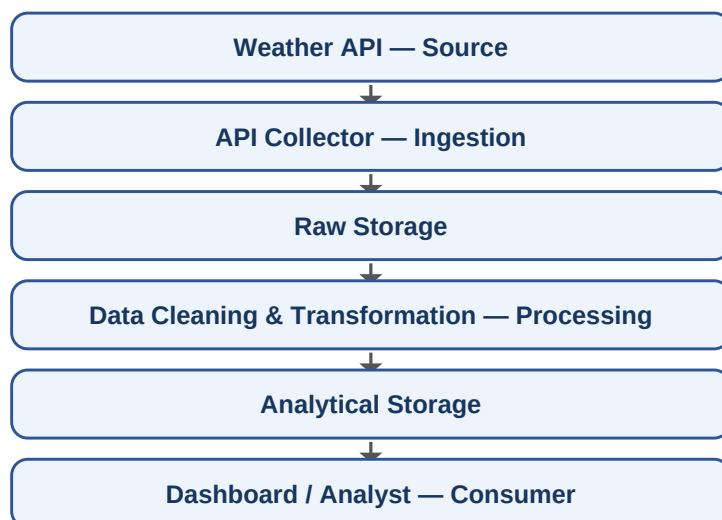
The selected source is a **Weather API**. It provides weather information for a requested location, such as temperature, humidity, wind speed, atmospheric pressure, weather condition, timestamp, and location.

Example data fields:

- City / location
- Temperature
- Humidity
- Wind speed
- Atmospheric pressure
- Weather condition
- Date and time

2. Diagram 1 — Logical Architecture

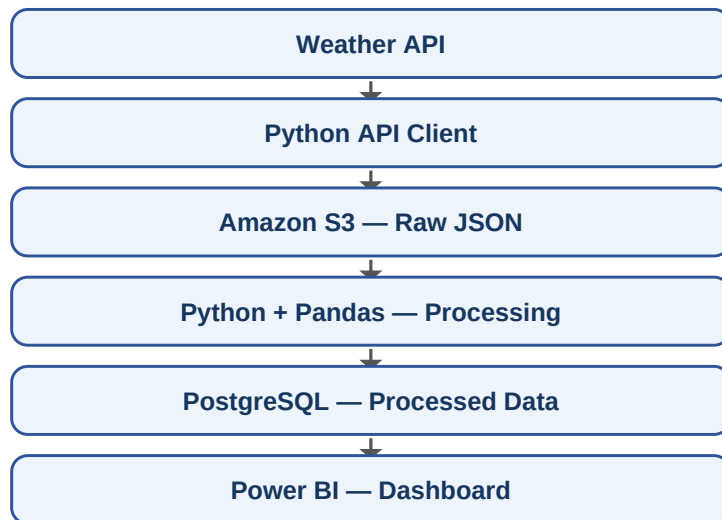
The logical architecture focuses on what each stage does rather than on specific vendor products.



Logical flow: Source → Ingestion → Raw Storage → Processing → Analytical Storage → Consumer.

3. Diagram 2 — Technology Mapping

The logical components are mapped to practical technologies. These choices are intentionally simple and are not intended to be the only possible solution.



Technology flow: Weather API → Python → Amazon S3 → Pandas → PostgreSQL → Power BI.

4. Component Explanation

Component	Why it exists
Weather API	Provides the source weather data such as temperature, humidity, wind speed, and conditions.
Python API Client	Calls the API periodically and collects the required weather records.
Amazon S3	Stores the original raw API responses so the source data is preserved before transformation.
Pandas	Cleans and transforms the raw data into a consistent, structured format.
PostgreSQL	Stores processed weather records in structured tables for efficient querying.
Power BI	Uses the processed data to create dashboards and visualizations for users and analysts.

5. Processing Stage

The processing stage converts the raw API response into clean and useful analytical data.



For example, a temperature value received as text can be converted to a numeric value. Duplicate records can also be removed before the final data is stored.

6. Question Log

1. Why can't the dashboard directly use the Weather API?

It could, but storing the data first allows historical analysis and reduces dependence on the API whenever a report is opened.

2. Why do we need raw storage?

Raw storage preserves the original source data. If processing logic changes or fails, the original records can be processed again.

3. Why is a processing stage required?

API data may contain missing values, duplicate records, inconsistent formats, or fields that are not needed for analysis.

4. Why use PostgreSQL after storing data in S3?

S3 is suitable for raw file storage, while PostgreSQL provides structured tables that are convenient for querying and dashboard access.

5. Why not store everything directly in PostgreSQL?

Keeping raw responses separately preserves the original data and avoids coupling raw source files to the final database structure.

6. Does the pipeline need to run continuously?

Not necessarily. Weather data can be collected periodically, for example every 30 minutes or every hour, depending on the application's requirements.

7. Short Architecture Explanation

The pipeline starts with the Weather API, which is the source of the data. A Python program periodically calls the API and collects weather information. The collected response is first stored as raw data in Amazon S3 so that the original source data is preserved.

The raw data is then processed using Pandas. During processing, invalid records, duplicates, and missing values are handled, and the data is converted into a consistent format. The cleaned data is stored in PostgreSQL, where it can be queried efficiently.

Finally, Power BI connects to the processed data and presents dashboards showing information such as temperature trends, humidity, wind speed, and weather conditions.

Why does data move from one component to the next?

Each stage has a clear responsibility: the API produces data, Python collects it, S3 preserves the raw data, Pandas prepares it for analysis, PostgreSQL stores structured records, and Power BI presents the results to users. This separation keeps the pipeline simple and understandable.

8. Final Architecture Summary



Final pipeline: Weather API → Python → Amazon S3 → Pandas → PostgreSQL → Power BI → Users.

The architecture is deliberately simple. Technologies such as Kafka, Spark, or Airflow are not included because the assignment does not require unnecessary complexity for this use case.